Referee Report on the ALCI_RCC5 Lean Development Rounds 26–28 Cold Review

Adversarial technical review of `Round19Transport.lean`

July 15, 2026

Abstract

This report reviews the rounds 26–28 changes in the supplied Lean 4 artifact `Round19Transport.lean`, using `ALCI_RCC5_REFERENCE.md` and the round-14 review findings as the yardstick. The question is not whether the Lean proofs typecheck; it is whether the statements now express a genuine decision-grade, arbitrary-domain, finite-certificate account of ALCI_RCC5 satisfiability. My verdict is **gap**. Round 27 genuinely repairs the old occurrence-only satisfiability interface. Round 26 is a correct finite-search reduction relative to an honest bounded Boolean checker, but the stated `Nonempty BoundedDecider` premise can be inhabited by an oracle that calls `Satisfiable`. Round 28 provides useful local tabulation lemmas and total decoders, but it does not replace the higher-order completeness witness with finite syntax, does not finite-code the labelling τ, and does not wire finite codes into the bounded checker. Therefore F1 is not closed and F3 is only partially reduced.

1 Scope and basis

I reviewed the supplied archive `cold_review_round28.zip`, in particular `Round19Transport.lean`. Line numbers in this report refer to that file as extracted from the archive. The review accepts the primer assumptions in the prompt: finite certificates may unfold to infinite generated models; `EQ` has strong identity semantics; inverse roles are absorbed; `Occ` is the generated occurrence type; and after round 27 `Satisfiable` is carrier-polymorphic.

The target is the interface layer: whether the new statements genuinely close the round-14 findings F1–F3. The key distinction is between a semantically true proposition in Lean and an honest finite decision procedure for the reference logic. A field of type `Concept -> Nat -> Bool` is Boolean-valued, but unless its definition is restricted to finite syntactic verification it can still encode non-finite or semantic information.

2 Executive verdict

Overall verdict: gap, repairable but not cosmetic.

Issue	Status	Reason
F2	Fixed	`Satisfiable` now quantifies over an arbitrary carrier `α : Type`; the soundness pipeline instantiates `α = Occ` as a genuine reference model.
F3	Partially fixed	`BoundedDecider.decidable` is a correct finite-search reduction for an honest checker, but `Nonempty BoundedDecider` allows an oracle checker.
F1	Not fixed	The normative completeness obligation still quantifies over `Catalog`, `Template`, function field `F`, and labelling `τ : Occ -> List Concept`; finite codes are not integrated.
Global conditional decidability	Overclaimed	The current development does not yet justify "decidable modulo exactly F6 □ W2'". Additional interface and finite-coding obligations remain.

3 Finding 1: `BoundedDecider` can smuggle satisfiability

Severity: Critical. **Location:** `BoundedDecider`, `BoundedDecider.decidable`, `decidableSat_of_nonemp` lines 4567–4604.

The round-26 interface is:

```
structure BoundedDecider where
  bound : Concept → Nat
  check : Concept → Nat → Bool
  sound : ∀ C0 n, check C0 n = true → Satisfiable C0
  complete : ∀ C0, Satisfiable C0 → ∃ n, n < bound C0 ∧ check C0 n = true
```

The theorem `BoundedDecider.decidable` is correct relative to such a structure. It searches `List.range (D.bound C0)` and uses `sound` and `complete` to connect the Boolean search result with `Satisfiable C0`. That is the right proof shape for a bounded certificate checker.

The defect is that the structure itself does not enforce that `check` is finite, syntactic, executable in the intended sense, or independent of the semantic predicate being decided. In Lean, `check : Concept -> Nat -> Bool` is an arbitrary function. With classical choice, it can simply query `Satisfiable C`. The following witness illustrates the problem:

```
namespace Round19

noncomputable def oracleBoundedDecider : BoundedDecider := by
  classical
  refine
    { bound := fun _ => 1
      check := fun C _ => if Satisfiable C then true else false
      sound := ?sound
      complete := ?complete }
  · intro C n hcheck
    by_cases h : Satisfiable C
    · exact h
    · have hfalse : False := by simpa [h] using hcheck
      cases hfalse
```

```
  · intro C hsat
    refine ⟨0, by decide, ?_⟩
    simp [hsat]

noncomputable example : Nonempty BoundedDecider :=
  ⟨oracleBoundedDecider⟩

end Round19
```

This witness ignores the candidate code `n`; the semantic answer is stored directly in `check`. It satisfies the stated `sound` and `complete` fields. Therefore `Nonempty BoundedDecider -> forall C0, Decidable (Satisfiable C0)` is not an honest algorithmic premise as currently stated. It proves decidability from the existence of a Boolean oracle, not from a finite syntactic certificate scheme.

This does not make `BoundedDecider.decidable` useless. It is a good target theorem once `check` is replaced by, or derived from, a first-order finite-code parser and verifier. But as an answer to F3, the current `Nonempty` theorem is too permissive.

4 Finding 2: finite coding is not wired into the completeness obligation

Severity: Critical. **Location:** `Template`, `Cert`, `Catalog`, `CompletenessObligation`, `FinTemplate`, `FinCatalog`; lines 224–241, 3311–3317, 4460–4477, 4659–4695.

The old higher-order certificate objects remain the normative objects:

```
structure Template where
  nSlots : Nat
  nFresh : Nat
  net : TPort → TPort → Atom

structure Cert where
  nCore : Nat
  coreNet : Nat → Nat → Atom
  templates : List Template
  steps : List Step
  f : Nat → (Nat → Atom) → (Nat → Atom) → Nat → Atom

structure Catalog where
  nCore : Nat
  coreNet : Nat → Nat → Atom
  templates : List Template
  root : Nat
  attaches : List Attach
  F : Nat → (Nat → Atom) → (Nat → Atom) → Nat → Atom
```

The round-25 completeness obligation still quantifies over these higher-order witnesses:

```
def CompletenessObligation : Prop :=
  ∀ C0 : Concept, Satisfiable C0 →
    ∃ (K : Catalog) (plan : List (Nat × Nat))
      (τ : Occ → List Concept) (root : Occ),
      CatOk K ∧ ... ∧
```

```
      PlanOk K plan ∧ SCat (buildCert K plan) ∧
      Hintikka ... τ ∧
      root ∈ existingBefore (buildCert K plan)
        (buildCert K plan).steps.length ∧
      C0 ∈ τ root
```

Round 28 introduces `FinTemplate` and `FinCatalog`, but no theorem replaces or bridges this obligation with a finite-code obligation. There is no theorem of the form:

```
∀ C0, Satisfiable C0 →
  ∃ code : Nat, check C0 code = true
```

nor a structured bridge of the form:

```
∀ K plan τ root,
  oldCompletenessWitness K plan τ root →
  ∃ FK finiteTauCode,
    decodedFiniteWitness FK finiteTauCode plan root
```

This is not merely engineering wiring. The completeness theorem still speaks the old higher-order language, while the decision theorem speaks an unconstrained Boolean checker. The finite-code objects sit between them but are not connected to either endpoint.

5 Finding 3: `fn_finitely_coded` is true but too weak for F1

Severity: Major. **Location:** `lookupT`, `lookupT_tableOf`, `fn_finitely_coded`; lines 4623–4653.

The theorem states:

```
theorem fn_finitely_coded {β γ : Type} [DecidableEq β] (f : β → γ)
    (d : γ) (l : List β) :
    ∃ t : List (β × γ), ∀ b ∈ l, lookupT d t b = f b
```

This is a correct local tabulation lemma: given a finite list `l`, one can store the values of `f` on `l` in an association list.

It does not prove that the certificate language is finite syntax. The table is constructed by evaluating the original function `f`. If `f` was a higher-order oracle field, the theorem merely copies finitely many oracle answers into a list. For F1, the missing result is global: all function values inspected by `CatOk`, `SCat`, `PlanOk`, `buildCert`, `Hintikka`, and `sat_from_hintikka` must be drawn from computably enumerable finite domains, and decoding the finite tables must preserve the required obligations.

Thus `fn_finitely_coded` is a useful lemma, not a closure of F1.

6 Finding 4: finite decoders are total, but no global faithfulness theorem exists

Severity: Major. **Location:** `FinTemplate.decode`, `FinCatalog.decode`; lines 4665–4695.

The decoders are total and do produce genuine Lean `Template` and `Catalog` values:

```
def FinTemplate.decode (T : FinTemplate) : Template where
  nSlots := T.nSlots
  nFresh := T.nFresh
  net := fun p q => lookupT Atom.dr T.netTable (p, q)

def FinCatalog.decode (K : FinCatalog) : Catalog where
  nCore := K.nCore
  coreNet := fun i j => lookupT Atom.dr K.coreNetTable (i, j)
  templates := K.templates.map FinTemplate.decode
  root := K.root
  attaches := K.attaches
  F := fun i r1 r2 j =>
    lookupT Atom.dr K.fTable
      (i, fnToList K.nCore r1, fnToList K.slotBound r2, j)
```

Defaulting missing entries to `Atom.dr` is fine for a total decoder. The problem is that total decoding is harmless only if all inspected entries are present and faithfully tabulated. The file does not prove decoded finite catalogues preserve the actual obligations used by the pipeline, such as `CatOk`, `SCat`, `PlanOk`, `buildCert` faithfulness, `Hintikka`, or root membership and satisfaction.

Local faithfulness on a supplied finite domain is not enough. A real closure theorem would identify the inspected domain from the certificate/checker and prove that every query made by the pipeline lands inside the tabulated part.

7 Finding 5: `f_factors_through_rows` is real but only pointwise

Severity: Major. **Location:** `CatOk.F_reads`, `f_factors_through_rows`; lines 3510–3513 and 4728–4741.

The theorem correctly formalizes a row-read principle:

```
theorem f_factors_through_rows
    (F : Nat → (Nat → Atom) → (Nat → Atom) → Nat → Atom)
    (nCore : Nat) (nSlots : Nat)
    (hreads : ∀ i r1 r1' r2 r2' j,
      (∀ c, c < nCore → r1 c = r1' c) →
      (∀ k, k < nSlots → r2 k = r2' k) →
      F i r1 r2 j = F i r1' r2' j)
    (i : Nat) (r1 r2 : Nat → Atom) (j : Nat) :
    F i r1 r2 j
      = F i (fun c => (fnToList nCore r1).getD c Atom.dr)
            (fun k => (fnToList nSlots r2).getD k Atom.dr) j
```

This is not fake; it is the right extensional idea. However, it is only a pointwise factoring lemma. It does not construct the finite `fTable`; does not bound `i` or `j`; does not enumerate possible atom rows; and does not connect the global `slotBound` in `FinCatalog` with every template arity in `K.templates`. The corresponding `CatOk.F_reads` clause is template-indexed:

```
F_reads : ∀ t r1 r1' r2 r2' j,
  (∀ c, c < K.nCore → r1 c = r1' c) →
  (∀ k, k < (K.tmpl t).nSlots → r2 k = r2' k) →
  K.F t r1 r2 j = K.F t r1' r2' j
```

Round 28 moves toward a finite key for `F`, but it does not yet show that the finite key space covers all `F` calls made by the generated certificates.

8 Finding 6: `certK_finitely_codable` is degenerate

Severity: Major. **Location:** `certK_finitely_codable`; lines 4745–4754.

The theorem is advertised as showing that `certK`'s catalogue data is finitely codable and that decoding reproduces `certK`'s core net and template nets on inspected domains. The actual statement is:

```
theorem certK_finitely_codable :
    ∃ FK : FinCatalog,
      FK.nCore = certK.nCore ∧
      (∀ i j, (i, j) ∈ ([] : List (Nat × Nat)) →
        (FinCatalog.decode FK).coreNet i j = certK.coreNet i j) := by
  refine ⟨⟨certK.nCore, [], [], certK.root, certK.attaches, 0, []⟩, rfl, ?_⟩
  intro i j h; cases h
```

The inspected domain is the empty list. The chosen finite catalogue has no templates and no tables. Since `certK` itself has two templates (lines 3998–4000), the witness does not even preserve template count:

```
def FK_emptyForCertK : FinCatalog :=
  { nCore := certK.nCore
    coreNetTable := []
    templates := []
    root := certK.root
    attaches := certK.attaches
    slotBound := 0
    fTable := [] }

example : (FinCatalog.decode FK_emptyForCertK).templates.length = 0 := rfl
example : certK.templates.length = 2 := rfl

example :
    (∀ i j, (i, j) ∈ ([] : List (Nat × Nat)) →
      (FinCatalog.decode FK_emptyForCertK).coreNet i j = certK.coreNet i j) := by
  intro i j h
  cases h
```

This is a red flag for the honesty of the non-vacuity claim. Because `certK.nCore = 0`, an empty core-pair table is not itself suspicious; the suspicious part is the theorem name and comment, which claim reproduction of catalogue data and template nets but prove only vacuous core-net agreement over an empty domain.

9 Finding 7: the labelling τ remains higher-order

Severity: Major. **Location:** `CompletenessObligation`, `Hintikka`; lines 4158–4167 and 4460–4477.

The completeness witness still includes:

```
τ : Occ → List Concept
```

A decision procedure needs a finite representation of the labelling information relevant to `C0`. The usual target would be a finite type assignment over closure concepts, perhaps with a blocking or regular representation for the finite generator. Round 28 does not introduce such a code and does not connect `BoundedDecider.check` to verification of a finite labelling.

This is independent of the catalogue-field coding issue. Even if `Template.net`, `Catalog.coreNet`, and `Catalog.F` were fully tabulated, the current completeness obligation would still quantify over a function `Occ -> List Concept`.

10 Finding 8: Round 27 genuinely fixes the old carrier defect

Severity: Positive finding. Location: `Interp`, `sat`, `RCC5Interp`, `Satisfiable`, `certInterp`, `sat_from_hintikka`; lines 4132–4146, 4209–4214, 4224–4300.

Round 27 is the strongest part of the fix. The new interpretation type is carrier-polymorphic:

```
structure Interp (α : Type) where
  dom : α → Prop
  rho : α → α → Atom
  val : Nat → α → Prop
```

Satisfaction quantifies over domain elements in the modal clauses:

```
| x, .ex r c => ∃ y, I.dom y ∧ I.rho x y = r ∧ sat I y c
| x, .all r c => ∀ y, I.dom y → I.rho x y = r → sat I y c
```

The RCC5 model condition enforces reflexive `EQ`, strong `EQ` as identity, converse coherence, and composition closure on domain elements:

```
structure RCC5Interp {α : Type} (I : Interp α) : Prop where
  refl_eq : ∀ x, I.dom x → I.rho x x = Atom.eq
  eq_id : ∀ x y, I.dom x → I.dom y → I.rho x y = Atom.eq → x = y
  conv_ : ∀ x y, I.dom x → I.dom y → I.rho y x = conv (I.rho x y)
  comp_ : ∀ x y z, I.dom x → I.dom y → I.dom z →
    I.rho x z ∈ comp (I.rho x y) (I.rho y z)
```

The new satisfiability predicate is:

```
def Satisfiable (C0 : Concept) : Prop :=
  ∃ (α : Type), ∃ I : Interp α,
    RCC5Interp I ∧ ∃ x, I.dom x ∧ sat I x C0
```

This no longer fixes the carrier to `Occ`. The soundness pipeline still produces a genuine reference model by choosing `α = Occ`:

```
theorem sat_from_hintikka ... : Satisfiable C0 :=
  ⟨Occ, certInterp C τ, certInterp_rcc5 C hwf hs τ, root, hroot,
    truth_lemma _ _ τ H C0 root hroot hC0⟩
```

I do not see the old F2 defect after this refactor. If one wanted maximum universe generality, the carrier could be parameterized as `Type u`; however, for the stated reference semantics this is not the Occ-only under-approximation that F2 identified.

11 Finding 9: `satisfiable_iff_hintikkaP` remains meaningful but is not finite completeness

Severity: Minor / explanatory. **Location:** `satisfiable_iff_hintikkaP`; lines 4398–4406.

The theorem states:

```
theorem satisfiable_iff_hintikkaP (C0 : Concept) :
    Satisfiable C0 ↔
      ∃ (α : Type), ∃ I : Interp α,
        RCC5Interp I ∧ ∃ τ, HintikkaP I τ ∧ ∃ x, I.dom x ∧ τ x C0
```

The forward direction sets `τ x C := sat I x C`; the reverse direction uses the polymorphic truth lemma. This is tautological in the good sense: the Hintikka abstraction loses no semantic information. It remains meaningful after the refactor.

It should not, however, be counted as finite certificate completeness. It supplies neither a finite carrier, nor a finite type code, nor a bounded search space.

12 Finding 10: the F4 ledger remains unresolved

Severity: Moderate. **Location:** deferred `SCat.net_r3` over-check.

The prompt states that F4 was verified and repair deferred. Rounds 26–28 do not repair it. Therefore the global claim should not say the only remaining obstacles are exactly F6 □ W2′ unless F4 has been shown irrelevant to the completeness route or explicitly excluded from the current claim. This point is secondary to the F1/F3 gaps, but it matters for the advertised obstacle ledger.

13 Direct answers to the review questions

1. `BoundedDecider.decidable` genuinely derives `Decidable (Satisfiable C0)` from a supplied `D`; however, `D` may smuggle the answer through `check`.
2. `sound` and `complete` are the right semantic obligations for a bounded Boolean characterization. They are not enough to ensure an honest computable checker because `check` is an arbitrary function.
3. `Nonempty BoundedDecider` is not an honest algorithmic premise. The oracle witness above has `bound = 1` and `check C n = if Satisfiable C then true else false`.
4. `Satisfiable` is genuinely arbitrary-domain in the relevant reference-semantics sense. The old `Occ`-only defect is fixed.
5. The soundness pipeline still produces a genuine reference model by instantiating `α = Occ`; the polymorphic refactor did not silently weaken the soundness theorem.
6. `satisfiable_iff_hintikkaP` is meaningful as semantic abstraction completeness, but it is not finite-certificate completeness.
7. `fn_finitely_coded` is correct but local: it tabulates a function on a supplied finite list. It does not itself finite-code the certificate language.
8. `FinTemplate.decode` and `FinCatalog.decode` are total and produce genuine old-style objects. The faithfulness theorems are only local, and `certK_finitely_codable` is degenerate.
9. `f_factors_through_rows` is a real consequence of row-read discipline, but it is not a finite `F`-table construction and does not bound all keys used by generated certificates.

10. Calling F1 “substantially closed” is overclaiming. Real closure requires restating or bridging the completeness/decision pipeline over finite codes, including finite labellings.
11. The collective conditional-decidability claim is not yet honest as “modulo exactly F6 □ W2′”. Remaining issues include non-oracular checking, finite-code integration, finite labelling, and the deferred F4 over-check.
12. The main bad interaction is that round 26’s `Nat`-code Boolean checker is not connected to round 28’s finite codes. The checker can be an oracle while the finite-code structures remain unused by the decision theorem.

14 What would close the gap

A convincing repair would include the following formal interface changes.

1. Replace the bare `check : Concept -> Nat -> Bool` premise, or derive it, from a first-order parser and verifier for finite witness codes. The verifier should not mention `Satisfiable` or any semantic oracle.
2. State the completeness obligation directly over finite witness codes, or prove a bridge from old `Catalog`/`Template`/`τ` witnesses to finite codes that preserve every predicate consumed by `sat_from_hintikka` and the catalogue soundness theorem.
3. Finite-code `τ`. For a decision procedure this is at least as important as finite-coding `coreNet`, `net`, and `F`.
4. Prove global inspected-domain coverage: every lookup performed by `CatOk`, `SCat`, `buildCert`, `pairVal`, `Hintikka`, and the final satisfaction witness must be inside the decoded tables.
5. Replace `certK_finitely_codable` with a non-degenerate theorem preserving the actual template list, core net on the relevant core square, template nets on all relevant ports, and the steering table on all relevant row keys.

15 Final assessment

Round 27 is adequate and closes F2. Round 26 gives a useful target theorem for a future honest checker, but its `Nonempty BoundedDecider` theorem is vulnerable to exactly the oracle-smuggling objection that a decision-grade interface must exclude. Round 28 supplies local finite-tabulation ingredients but does not close F1: the main completeness witness remains higher-order, the finite-code layer is not used by the decision theorem, and the non-vacuity theorem is degenerate.

Accordingly, the correct status is **gap**. The fixes are repairable, but the missing work is not merely cosmetic or implementation-level. It requires a formal bridge from finite syntactic codes to the existing soundness/completeness pipeline, plus an explicitly non-oracular bounded checker.

A Witness snippets

The following snippets are provided in the accompanying `supporting_witnesses.lean` file. They are intended to be placed next to `Round19Transport.lean` and imported with `import Round19Transport`. The local execution environment used to prepare this report did not have Lean installed, so these snippets were not kernel-run here. Their role in the

report is to expose the weakness of the statements, not to re-check the artifact's existing proofs.

```
import Round19Transport

namespace Round19

noncomputable def oracleBoundedDecider : BoundedDecider := by
  classical
  refine
    { bound := fun _ => 1
      check := fun C _ => if Satisfiable C then true else false
      sound := ?sound
      complete := ?complete }
  · intro C n hcheck
    by_cases h : Satisfiable C
    · exact h
    · have hfalse : False := by simpa [h] using hcheck
      cases hfalse
  · intro C hsat
    refine ⟨0, by decide, ?_⟩
    simp [hsat]

noncomputable example : Nonempty BoundedDecider :=
  ⟨oracleBoundedDecider⟩

def FK_emptyForCertK : FinCatalog :=
  { nCore := certK.nCore
    coreNetTable := []
    templates := []
    root := certK.root
    attaches := certK.attaches
    slotBound := 0
    fTable := [] }

example : (FinCatalog.decode FK_emptyForCertK).templates.length = 0 := rfl
example : certK.templates.length = 2 := rfl

example :
    (∀ i j, (i, j) ∈ ([] : List (Nat × Nat)) →
      (FinCatalog.decode FK_emptyForCertK).coreNet i j = certK.coreNet i j) := by
  intro i j h
  cases h

end Round19
```